

```

    }
    cout << "\nTotal Marks: " << total << "/" << Tmax << endl;
};

int main() {
    Student s;
    int marks[5] = {85, 90, 78, 88, 92}; // Sample marks
    s.assign("Ananya", marks); // Assigning values
    s.display(); // Displaying student info
    s.compute(); // Calculating average
    return 0;
}

```

Syntax Breakdown:

Syntax	What it Does
<code>`class ClassName {}`</code>	Declares a class
<code>`string sname;`</code>	A string variable for student name
<code>`int marks[5];`</code>	Array to store marks
<code>`void function()`</code>	Declares a member function
<code>`object.function()`</code>	Calls the function using the object
<code>`for (int i = 0; i < 5; i++)`</code>	Looping through marks

Let me know if I should continue with **Program 2: Operator & Function Overloading** in the same format!



give all 14

Awesome. I'll now give you **all 14 programs**, one by one, with the following for each:

- ✓ Topic/Concept explanation
- ✓ Full executable C++ code with ``main()``
- ✓  Syntax explanations in comments and tables

Let's roll!

✓ Program 1: Class ``STUDENT`` with `assign()`, `compute()`, `display()`

Concept:

This shows basic **class creation**, **methods**, and working with arrays inside a class.

```

cpp

#include <iostream>
#include <string>
using namespace std;

```

```
class Student {
    string sname;
    int marks[5];
    int total;
    int Tmax;

public:
    void assign(string name, int m[]) {
        sname = name;
        total = 0;
        Tmax = 500;
        for (int i = 0; i < 5; i++) {
            marks[i] = m[i];
            total += m[i];
        }
    }

    void compute() {
        float average = total / 5.0;
        cout << "Average Marks: " << average << endl;
    }

    void display() {
        cout << "Name: " << sname << endl;
        cout << "Marks: ";
        for (int i = 0; i < 5; i++)
            cout << marks[i] << " ";
        cout << "\nTotal: " << total << "/" << Tmax << endl;
    }
};

int main() {
    Student s;
    int marks[5] = {80, 85, 78, 90, 88};
    s.assign("Riya", marks);
    s.display();
    s.compute();
    return 0;
}
```

✓ Program 2: Operator Overloading & Function Overloading

Concept:

- Operator Overloading allows us to redefine operators (`+`, `-`, etc.) for user-defined types.
- Function Overloading = Same function name, different parameters.

```
cpp

#include <iostream>
using namespace std;

class Complex {
    int real, imag;

public:
    Complex(int r = 0, int i = 0) {
        real = r;
        imag = i;
    }

    // Overload '+' operator
    Complex operator + (Complex const &obj) {
        return Complex(real + obj.real, imag + obj.imag);
    }

    // Function Overloading
```

```
void display() {
    cout << "Real: " << real << ", Imag: " << imag << endl;
}

void display(string label) {
    cout << label << " => Real: " << real << ", Imag: " << imag << endl;
}

};

int main() {
    Complex c1(3, 2), c2(1, 7);
    Complex c3 = c1 + c2;

    c1.display("C1");
    c2.display("C2");
    c3.display("Sum");
    return 0;
}
```

✓ Program 3: Friend Function and Friend Class

Concept:

- A **friend function** or **friend class** can access private members of another class.

```
cpp

#include <iostream>
using namespace std;

class Box;

class Display {
public:
    void show(Box &);
};

class Box {
    int length;

public:
    Box(int l) { length = l; }

    // Declare friend class
    friend class Display;
};

void Display::show(Box &b) {
    cout << "Length of box: " << b.length << endl;
}

int main() {
    Box b(10);
    Display d;
    d.show(b);
    return 0;
}
```

✓ Program 4: Single Inheritance

Concept:

A derived class inherits from a base class.

```
cpp

#include <iostream>
using namespace std;

class Animal {
public:
    void eat() {
        cout << "Eating..." << endl;
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Barking..." << endl;
    }
};

int main() {
    Dog d;
    d.eat();
    d.bark();
    return 0;
}
```

✓ Program 5: Multiple Inheritance

Concept:

One class inherits from **more than one base class**.

```
cpp

#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B {
public:
    void showB() {
        cout << "Class B" << endl;
    }
};

class C : public A, public B {
public:
    void showC() {
        cout << "Class C" << endl;
    }
};

int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();
    return 0;
}
```

✓ Program 6: Multilevel Inheritance

🔍 Concept:

A class derived from a derived class (like Grandparent → Parent → Child).

```
cpp

#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "A" << endl;
    }
};

class B : public A {
public:
    void showB() {
        cout << "B" << endl;
    }
};

class C : public B {
public:
    void showC() {
        cout << "C" << endl;
    }
};

int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();
    return 0;
}
```

✓ Program 7: Constructor & Destructor

🔍 Concept:

- Constructor initializes an object automatically.
- Destructor frees resources when the object is destroyed.

```
cpp

#include <iostream>
using namespace std;

class Test {
public:
    Test() {
        cout << "Constructor called!" << endl;
    }
    ~Test() {
        cout << "Destructor called!" << endl;
    }
};

int main() {
```

```
Test t;
return 0;
}
```

✓ Program 8: THIS Pointer

Concept:

`this` is a pointer that refers to the current object.

```
cpp

#include <iostream>
using namespace std;

class Sample {
    int x;

public:
    void setX(int x) {
        this->x = x;
    }

    void display() {
        cout << "x = " << x << endl;
    }
};

int main() {
    Sample s;
    s.setX(100);
    s.display();
    return 0;
}
```

✓ Program 9: Virtual Function

Concept:

Virtual functions allow **runtime polymorphism** (late binding).

```
cpp

#include <iostream>
using namespace std;

class Base {
public:
    virtual void show() {
        cout << "Base class show()" << endl;
    }
};

class Derived : public Base {
public:
    void show() override {
        cout << "Derived class show()" << endl;
    }
};

int main() {
    Base *ptr;
```

```
Derived d;
ptr = &d;

ptr->show(); // Calls Derived's version
return 0;
}
```

✓ Program 10: Data Conversion Between Classes

Concept:

Shows how to convert data from one class to another using conversion functions.

```
cpp

#include <iostream>
using namespace std;

class B;

class A {
    int val;
public:
    A(int x) { val = x; }
    int getVal() { return val; }

    // Conversion function
    operator B();
};

class B {
    int data;
public:
    B() {}
    B(int d) { data = d; }

    void show() {
        cout << "Value in B: " << data << endl;
    }
};

// Define conversion
A::operator B() {
    return B(val);
}

int main() {
    A a(50);
    B b = a;
    b.show();
    return 0;
}
```

✓ Program 11: File Operations

Concept:

Write to and read from a file using `fstream`.

```
cpp

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream fout("data.txt");
    fout << "Hello File!";
    fout.close();

    ifstream fin("data.txt");
    string content;
    getline(fin, content);
    cout << "Read: " << content << endl;
    fin.close();
    return 0;
}
```

✓ Program 12: Pass and Return Object

Concept:

Demonstrates how to pass objects to functions and return objects.

```
cpp

#include <iostream>
using namespace std;

class Number {
    int x;
public:
    void set(int a) { x = a; }

    void display() { cout << "x = " << x << endl; }

    Number add(Number n) {
        Number temp;
        temp.x = x + n.x;
        return temp;
    }
};

int main() {
    Number n1, n2, result;
    n1.set(10);
    n2.set(20);
    result = n1.add(n2);
    result.display();
    return 0;
}
```

✓ Program 13: Unary Operator Overloading

```
cpp

#include <iostream>
using namespace std;

class Counter {
    int value;
```

```

public:
    Counter(int v = 0) { value = v; }

    void operator++() {
        ++value;
    }

    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Counter c(5);
    ++c;
    c.display();
    return 0;
}

```

✓ Program 14: Binary Operator Overloading

```

cpp

#include <iostream>
using namespace std;

class Point {
    int x;
public:
    Point(int val = 0) { x = val; }

    Point operator+(Point p) {
        return Point(x + p.x);
    }

    void display() {
        cout << "x = " << x << endl;
    }
};

int main() {
    Point p1(10), p2(20), p3;
    p3 = p1 + p2;
    p3.display();
    return 0;
}

```

Would you like a **PDF of this too**, with formatted explanations and diagrams? I can generate that next.



give me the pdf same content with black background and other colour text

It seems like I can't do more advanced data analysis right now. Please try again later.

The PDF with black background and colored text couldn't be generated at the moment due to a tool issue. If you'd like, I can try again once things are back up, or I can provide you a text version here in chat that you can style manually in Word or another editor. Let me know what works best for you.



cool

Awesome! Just ping me when you want to try generating that black-background PDF again or need help with anything else—from tweaking code to making fancy notes.

Also, if you'd like the styled version now in Word or Notion format (dark mode friendly), I can help you prep that too. Just say the word! 😎



is multiple and multilevel inheritance the same thing

Great question — and **no, multiple inheritance and multilevel inheritance are not the same** in C++. Here's a clear breakdown:

◆ 1. Multiple Inheritance

A class inherits from **more than one base class**.

📌 Syntax:

```
cpp
class A { };
class B { };
class C : public A, public B { }; // Class C inherits from A and B
```

📌 Example scenario:

- Class `Teacher` and `Researcher` are base classes.
- Class `Professor` inherits from both — combining both roles.

📌 Key point:

- Might cause **ambiguity** if both base classes have members with the same name (solved using scope resolution).

◆ 2. Multilevel Inheritance

A class is derived from a class which is already derived from another class. Like a **chain**.

📌 Syntax:

```
cpp
class A { };
class B : public A { };
class C : public B { }; // Class C indirectly inherits A through B
```

📌 Example scenario:

- ``LivingBeing` → `Animal` → `Dog``

📌 Key point:

- Shows a **hierarchy or lineage** from general to specific.

🧠 In Simple Terms:

Concept	Structure	Analogy
Multiple Inheritance	Child ← Parent1 + Parent2	A person with 2 job roles
Multilevel Inheritance	Child ← Parent ← Grandparent	A grandchild in a family hierarchy

Let me know if you'd like a small C++ code demo of both!



explain unary and binary op overloading

Sure! Here's a clear explanation of **Unary** and **Binary Operator Overloading** in C++:

⚙️ Operator Overloading Basics

Operator overloading means giving special meaning to an operator (like ``+``, ``-``, ``++``, etc.) for your own classes, so you can use them like built-in types.

1 Unary Operator Overloading

- **Unary operators** act on **one operand**.
- Examples: ``++``, ``--``, ``-`` (negation), ``!`` (logical NOT).

How to overload unary operator?

- Inside the class, you define a function using the keyword ``operator`` followed by the operator symbol.
- It takes **no arguments** (except for ``operator-()`` in some cases).

Example: Overloading unary ``++`` (prefix increment)

```
cpp
class Counter {
    int count;
public:
    Counter() : count(0) {}

    // Overload prefix ++ operator
    Counter operator++() {
        ++count;      // increment count
        return *this;  // return updated object
    }
}
```

```

    }

    void display() {
        std::cout << "Count: " << count << std::endl;
    }
};

int main() {
    Counter c;
    ++c; // Calls overloaded operator++
    c.display(); // Output: Count: 1
}

```

Notes:

- For **postfix** `++` (like `c++`), the operator function takes an `int` dummy parameter to differentiate.

2 Binary Operator Overloading

- **Binary operators** act on **two operands**.
- Examples: `+`, `-`, `\*`, `/`, `==`, etc.

How to overload binary operator?

- Inside the class, define a function `operator+` (or any other operator).
- It usually takes **one argument** (the right-hand side operand).
- The left-hand side operand is the object calling the function.

Example: Overloading binary `+`

```

cpp

class Point {
    int x, y;
public:
    Point(int a, int b) : x(a), y(b) {}

    // Overload + operator to add two Point objects
    Point operator+(const Point &p) {
        return Point(x + p.x, y + p.y);
    }

    void display() {
        std::cout << "(" << x << ", " << y << ")" << std::endl;
    }
};

int main() {
    Point p1(1, 2), p2(3, 4);
    Point p3 = p1 + p2; // Calls overloaded operator+
    p3.display();       // Output: (4, 6)
}

```

Summary

Operator Type	Operands	Function Signature Example	Use Case
Unary Operator	One operand	<code>`ReturnType operator++()`</code>	Increment, negate, logical NOT
Binary Operator	Two operands	<code>`ReturnType operator+(const T&)`</code>	Adding two objects

If you want, I can also share fully commented C++ programs for these! Would you like that?